

FENIXCMS_1

FASE 1 - CIERRE DEL APROVISIONAMIENTO GRATUITO Y PAGO PAYPAL REAL

****Especificacion tecnica para preparar FenixCMS para produccion****

Documento orientado a ejecutar la fase en AI Studio sobre el repositorio `fenixcms-spec/moreno`.

La prioridad es impedir que una peticion publica pueda crear Tenant, License, Domain o Membership sin una confirmacion real de pago.

Objetivo de salida

La venta publica debe seguir este flujo:

1. Crear orden de PayPal.
2. Usuario aprueba.
3. Capturar en servidor.
4. Verificar el resultado real.
5. Aprovisionar una sola vez.
6. Dejar trazabilidad completa.

****Estado de partida auditado:**** el codigo actual permite que `/api/tenants/provision` llegue al aprovisionamiento sin demostrar primero un pago PayPal capturado. Ademas, `SaasLanding` llama directamente a esa ruta y `saas-checkout.service` conserva URLs simuladas y verificaciones debiles.

Esta fase corrige ese camino antes de publicar la venta.

1. Ficha de la fase

| Elemento | Contenido |

|---|---|

| Fase | FASE 1 - Cierre del aprovisionamiento gratuito y pago PayPal real |

| Ejecutar | AI Studio / repositorio FenixCMS. No desplegar al VPS hasta completar los criterios de aceptacion |

| Prioridad | CRITICA - bloqueo de produccion para venta publica |

| Resultado | Pago validado por PayPal en servidor y aprovisionamiento atomico e idempotente |

| No hacer en esta fase | No rediseñar todo el CMS, no cambiar funcionalidades de storefront no relacionadas, no meter datos de prueba como sustituto del pago real |

| Archivos principales | `app/api/tenants/provision/route.ts`, `components/saas/SaaS Landing.tsx`, `lib/services/saas-checkout.service.ts`, `lib/services/webhook.service.ts`, `app/api/webhooks/paypal/route.ts`, servicios Payment/Tenant/License/Subscription/Invoice, `prisma/schema.prisma` |

2. Bloqueo actual que se debe eliminar

- No debe existir ningun POST publico capaz de crear un Tenant o License simplemente porque el cliente envia `applicationId`, `planId`, `paymentProvider` o `transactionId`.
- Un `transactionId` recibido desde el navegador no es prueba de pago capturado. El servidor debe consultar/confirmar el estado real de la orden/captura con PayPal.
- El codigo no debe fabricar `checkoutUrl` de PayPal con token=`sessionId` ni marcar pagos como verificados solo porque el identificador no empieza por `fake\_` o `mock\_`.
- El aprovisionamiento no puede continuar despues de un fallo de una etapa critica. Debe existir una transaccion de base de datos o una maquina de estados que impida dejar recursos parcialmente creados.
- El webhook debe complementar el flujo y ser idempotente; nunca debe aceptar una firma solamente por formato o porque `certUrl` parezca pertenecer a PayPal.

3. Contrato de seguridad que debe quedar implementado

****Regla central: NO PAYMENT CAPTURED = NO PROVISIONING.****

Esta regla debe cumplirse aunque el cliente manipule el body, aunque llame directamente al endpoint, aunque repita la peticion y aunque intente reutilizar una session antigua.

| Invariant | Condicion obligatoria |

|---|---|

| Identidad del pago | PayPal Order ID y Capture ID deben provenir del flujo real del servidor/PayPal |

| Importe | El importe final se obtiene del plan/tarifa server-side y se compara con la respuesta de PayPal |

| Moneda | La moneda de la orden debe coincidir con la configuracion validada del plan |

| Plan/aplicacion | El navegador no puede elevar plan, applicationId, cuotas o entitlements despues de crear la orden |

| Tenant | `tenantId` nunca se acepta del cliente como autoridad para aprovisionar; se genera server-side |

| Idempotencia | Una misma orden/capture no puede crear dos tenants, dos licencias ni dos suscripciones |

| Rollback | Si falla cualquier paso critico, la operacion no puede terminar en estado falso de exito |

4. FASE 1A - Cerrar /api/tenants/provision

****Objetivo:**** convertir el endpoint de provision en un endpoint que solo pueda actuar después de una evidencia de pago válida y asociada al producto que se está comprando.

Cambios requeridos

- Eliminar cualquier posibilidad de que `applicationId`, `planId`, `billingPeriod`, `amount` o `tenantId` enviados por el cliente determinen por sí solos el recurso que se provisiona.
- No aceptar `transactionId` como prueba suficiente. Recibir, como máximo, un PayPal `orderId`/`captureId` asociado a una sesión de checkout creada por el servidor.
- Antes de aprovisionar, cargar la sesión de checkout persistida y comprobar que esta pertenece al contexto comercial correcto: plan, application, periodo, moneda, importe y estado.
- Comprobar en servidor el estado real de PayPal mediante la API de Orders/Capture cuando el flujo de captura sea server-side, y aceptar solo un estado de captura completada.
- Comprobar que el PayPal orderId no está ya consumido para otro tenant/licencia.
- No crear User/Membership/Domain/License/Tenant si falla cualquiera de las comprobaciones anteriores.
- Retirar fallbacks tipo `tenant\_demo`, `tenant\_{\$timestamp}`, `plan\_pro` o `app\_ecommerce` de cualquier camino de producción de provisioning.

Respuesta HTTP esperada

| Caso | Respuesta |

|---|---|

| Sin autenticación de checkout/pago válido | 400/401/403 según contrato, sin mutaciones |

| Orden no pertenece a la sesión o importe no coincide | 409 o 422, sin mutaciones |

| Pago no capturado | 409, estado `PAYMENT\_PENDING` / `PAYMENT\_NOT\_CAPTURED` |

| Orden ya procesada | 200/409 idempotente según contrato; nunca duplicar recursos |

| Error interno de DB | 500 y rollback; nunca devolver `success=true` |

| Provision correcta | 201 (primera vez) o 200 (repetición idempotente), con referencia interna segura |

5. FASE 1B - Sustituir el checkout PayPal simulado

El componente de venta no debe crear directamente el tenant. Debe iniciar un checkout real y esperar la confirmación del servidor.

Flujo recomendado

1. Browser pide crear checkout para un plan seleccionado.
2. Servidor valida plan/application/billingPeriod contra PostgreSQL y genera un `CheckoutSession` persistente.
3. Servidor crea la PayPal Order con importe y moneda calculados server-side.
4. Servidor devuelve al browser solo la información necesaria para continuar el checkout.
5. Usuario aprueba PayPal.
6. Browser llama a un endpoint de captura server-side pasando el `orderId`.
7. Servidor captura la orden y valida que la captura está `COMPLETED` y que el importe/moneda coinciden.
8. Servidor ejecuta el aprovisionamiento atómico dentro de una operación idempotente.
9. Webhook PayPal se procesa de forma independiente para reconciliación y estados posteriores.
10. Browser recibe un resultado de compra ya respaldado por persistencia.

****No utilizar como solución final**** las URLs hardcodeadas de checkout ni los tokens inventados por `sessionId`.

6. FASE 1C - Integración real PayPal Orders API / Capture

La implementación debe quedar separada en un servicio de pasarela (por ejemplo `PayPalGatewayService`) y un servicio de negocio de checkout. No mezclar lógica HTTP de PayPal con la transacción de provisioning.

Responsabilidades del gateway

- Obtener access token OAuth2 con las credenciales configuradas en variables de entorno.
- Crear Order con purchase unit server-side, importe calculado por FenixCMS y una referencia interna estable.
- Persistir PayPal Order ID y estado de la orden antes de seguir el flujo del cliente.
- Capturar `/v2/checkout/orders/{ORDER-ID}/capture` en servidor.
- Normalizar la respuesta de PayPal a un modelo interno sin pasar objetos completos de PayPal por todo el dominio.
- Comprobar estado, currency, amount y referencias del merchant antes de declarar el pago válido.
- Aplicar timeouts, control de errores y no exponer client secret al navegador.

Variables de entorno

Mantener secretos exclusivamente en entorno de servidor. Debe existir una diferenciación explícita entre sandbox y production y nunca usar credenciales de prueba en production.

...

```
PAYPAL_CLIENT_ID
PAYPAL_CLIENT_SECRET
PAYPAL_BASE_URL
PAYPAL_WEBHOOK_ID
...
```

Comprobaciones minimas de una captura

- HTTP exitoso de PayPal.
- Order ID igual al solicitado.
- Capture status = `COMPLETED`.
- Valor capturado igual al valor esperado con precisión monetaria exacta.
- Currency code igual al esperado.
- Referencia merchant/session interna compatible con la sesión persistida.
- La orden no está ya consumida por otra operación de provisioning.

7. FASE 1D - Precisión monetaria y autoridad de PostgreSQL

La fase no debe dejar que el importe provenga del navegador. El precio final debe salir del registro server-side de plan/precio y debe persistirse antes de llamar a PayPal.

Requisito recomendado

- Evitar `Float` para dinero nuevo o migrado a la ruta comercial crítica; utilizar `Decimal`/Prisma Decimal cuando el esquema permita hacerlo sin romper compatibilidad.
- Comparar importes con precisión decimal, no con igualdad de float.
- Guardar `amountExpected`, `currencyExpected` y snapshot del plan en la `CheckoutSession`.
- No recalcular el precio de una sesión antigua desde el navegador; usar el snapshot persistido de la sesión.

8. FASE 1E - Aprovisionamiento atómico

Una vez confirmada la captura, el sistema debe crear o actualizar los recursos de negocio como una unidad consistente.

| Orden de operación | Regla |

|---|---|

- | 1. Identificar CheckoutSession | Debe ser única y persistida |
- | 2. Registrar Payment | Asociar provider=PAYPAL, orderId y captureId, con unicidad |
- | 3. Crear Tenant | Solo una vez; nombre/slug validados |
- | 4. Crear License | Vinculada al tenant, plan y aplicación comprados |
- | 5. Crear Subscription | Con periodo y estados coherentes |
- | 6. Crear Invoice | Con importe final y referencia del pago |
- | 7. Crear Domain inicial | Solo el dominio que realmente corresponda al tenant; no verificar dominios arbitrarios como activos |
- | 8. Crear Membership/User | Solo con datos validados, sin passwords temporales inseguros |
- | 9. Marcar CheckoutSession consumida | Guardar referencia del resultado |

Si una etapa falla, usar `transaction()` de Prisma para revertir las mutaciones transaccionales. Si una operación externa impide una transacción pura, separar el proceso en estados persistidos y aplicar compensación/idempotencia; nunca ocultar el error con `try/catch` y devolver éxito.

9. FASE 1F - PayPal Webhooks: verificación criptográfica e idempotencia

El endpoint `/api/webhooks/paypal` debe aceptar eventos solo después de una verificación real de autenticidad. El chequeo actual de cabeceras, hostname y formato Base64 no es suficiente.

Implementación obligatoria

- Leer el body bruto sin alterarlo antes de calcular cualquier verificación que lo requiera.
- Usar la verificación oficial de PayPal `verify-webhook-signature` o implementar exactamente el mecanismo criptográfico documentado por PayPal.
- Comprobar webhook ID, transmission id, timestamp, cert URL y firma con la información recibida y la configuración del merchant.
- Persistir event id/provider event id con índice único antes de ejecutar efectos secundarios.
- Un evento repetido debe devolverse como procesado/sin cambios duplicados.

- Los eventos de captura completada deben poder reconciliar el pago aunque el browser no finalice correctamente su peticion.
- Eventos APPROVED no deben equivaler por si solos a pago capturado; fulfillment se hace despues de captura completada.

Eventos que deben quedar contemplados

| Evento | Uso interno |

---|---

| `CHECKOUT.ORDER.APPROVED` | Estado pendiente de captura; no entregar licencia por si solo |

| `PAYMENT.CAPTURE.PENDING` | Payment pending; no fulfillment final |

| `PAYMENT.CAPTURE.COMPLETED` | Confirmacion de captura; activar/reconciliar provisioning |

| `PAYMENT.CAPTURE.DENIED` | Marcar pago fallido/denegado; no crear recursos |

| `CHECKOUT.PAYMENT-APPROVAL.REVERSED` | Reconciliar reverso y bloquear/ajustar acceso segun regla de negocio |

| `BILLING.SUBSCRIPTION.PAYMENT.SUCCEEDED` | Actualizar renovacion cuando corresponda; separado de compra inicial |

10. FASE 1G - Eliminar verificaciones falsas y fallbacks de produccion

Revisar de manera dirigida `saas-checkout.service.ts` y cualquier servicio llamado por el checkout.

- Eliminar `verifyGatewayTransaction` basado en listas de IDs `fake\_`/`mock\_`/`test\_fraud\_token` como mecanismo de autenticidad. Esos checks solo sirven para tests, nunca como verificacion real.
- Eliminar `checkoutUrl` simulada de Stripe/PayPal del camino de produccion.
- Eliminar `INITIAL\_PLANS` / `INITIAL\_APPLICATIONS` / `INITIAL\_TENANTS` como fuentes de verdad del checkout de produccion.
- Los `catch` que actualmente registran warning y continuan deben convertirse en errores reales cuando el paso es obligatorio.
- Los valores por defecto de email, comercio, tenant o plan deben desaparecer del camino de venta real; solo permanecer en tests aislados si son necesarios.
- Verificar que el frontend no mantiene un `buyLicenseWithPayPal()` local que pueda generar licencias o tenants que parezcan reales.

11. FASE 1H - Idempotencia y concurrencia

El usuario puede hacer doble click, recargar, recibir un retry del navegador o recibir el mismo webhook varias veces. El resultado debe ser un solo aprovisionamiento.

| Mecanismo | Requisito |

---|---

| Unique providerOrderId | No duplicar Payment/Checkout |

| Unique providerCaptureId | No registrar la misma captura como dos pagos |

| Unique checkout session | La sesion consumida debe tener estado terminal y referencia a su provisioning |

| Transactional locking | Evitar dos workers aprovisionando el mismo checkout al mismo tiempo |

| Webhook event unique | Mismo evento PayPal no genera dos side effects |

12. PROMPT COMPLETO PARA AI STUDIO

Copiar este bloque como una unica instruccion para ejecutar la fase.

...

FASE 1 - CIERRE DEL APROVISIONAMIENTO GRATUITO Y PAGO PAYPAL REAL

OBJETIVO

Modificar FenixCMS para que NUNCA se pueda crear un Tenant, License, Subscription, Invoice, Domain o Membership desde la venta publica sin un pago PayPal real y verificado server-side. La regla es: NO PAYMENT CAPTURED = NO PROVISIONING.

REGLAS OBLIGATORIAS

1. Audita primero y enumera los archivos exactos que intervienen.
2. No aceptes tenantId, planId, applicationId, amount o transactionId del navegador como autoridad. Usa PostgreSQL como fuente de verdad.
3. Elimina del camino de produccion cualquier provisioning directo desde SaasLanding.
4. Implementa checkout real con PayPal Orders API y captura server-side.
5. La orden PayPal debe crearse con importe y moneda calculados server-side a partir del plan persistido.
6. Persiste CheckoutSession antes de completar el flujo del cliente y guarda PayPal orderId.
7. Implementa capture server-side y acepta solo una captura COMPLETED cuyo importe, moneda y referencias coincidan con la sesion persistida.
8. Haz el provisioning atomico e idempotente. Una misma orden/capture nunca puede crear dos tenants ni dos licencias.
9. Elimina URLs simuladas de PayPal/Stripe y verificadores que consideren valido un ID solo porque no coincide con fake_/mock_.
10. Cambia los catch que ocultan fallos criticos por errores reales; nunca devolver success=true si el

provisioning no se completo.

11. Revisa /api/webhooks/paypal. Implementa verificación criptográfica real usando el mecanismo oficial de PayPal verify-webhook-signature o el método criptográfico documentado.
12. Guarda idempotencia de webhook en PostgreSQL y no dependas de memoria para producción.
13. Trata CHECKOUT.ORDER.APPROVED como pendiente de captura. El fulfillment final solo después de PAYMENT.CAPTURE.COMPLETED o una captura server-side equivalente y validada.
14. Revisa la precisión monetaria de la ruta crítica. No uses Float para comparar dinero de forma insegura.
15. Revisa storeContext y cualquier buyLicenseWithPayPal local: no debe poder fabricar una licencia/tenant real que contradiga PostgreSQL en producción.
16. No uses INITIAL_PLANS/INITIAL_APPLICATIONS/INITIAL_TENANTS como fuente de verdad del checkout de producción.
17. No añadas funcionalidades nuevas no relacionadas. Esta fase es exclusivamente seguridad del pago y provisioning.

ARCHIVOS MINIMOS A REVISAR

- app/api/tenants/provision/route.ts
- components/saas/SaaS Landing.tsx
- lib/services/saas-checkout.service.ts
- lib/services/webhook.service.ts
- app/api/webhooks/paypal/route.ts
- servicios Payment/Tenant/License/Subscription/Invoice relacionados
- prisma/schema.prisma y migraciones necesarias
- lib/storeContext.tsx

CRITERIOS DE ACEPTACION

- A. Una llamada directa a /api/tenants/provision sin pago validado no crea ningún registro.
- B. Manipular planId, applicationId, amount o tenantId no permite cambiar el producto comprado ni el tenant destino.
- C. Un pago PayPal COMPLETED crea exactamente un Tenant + License + Subscription + Invoice + recursos iniciales esperados, una sola vez.
- D. Repetir la captura o repetir el request no duplica recursos.
- E. Un pago PENDING, DENIED o inexistente no provisiona.
- F. Una firma webhook no valida criptográficamente no produce side effects.
- G. El mismo webhook repetido no duplica side effects.
- H. Un error de base de datos durante provisioning no termina en success=true.
- I. npm run build pasa y los tests relacionados pasan.
- J. Se documentan variables de entorno y pasos sandbox/production sin exponer secretos.

SALIDA OBLIGATORIA DE AI STUDIO

1. Lista de archivos modificados.
2. Resumen técnico de cada cambio.
3. Flujo final de compra paso a paso.
4. Migraciones Prisma creadas, si las hay.
5. Tests creados/modificados y su resultado exacto.
6. Resultado exacto de npm run build.
7. Riesgos pendientes que NO hayan podido resolverse.
8. Confirmación explícita de que no existe un camino público para provisionar sin pago capturado.
9. No afirmes PASS si una prueba no fue ejecutada realmente.

...

13. Matriz de pruebas obligatorias antes de cerrar

ID	Prueba	Resultado esperado
---	---	---
P01	POST directo a `/api/tenants/provision` sin checkout valido	Bloqueado; 0 tenants nuevos
P02	Manipular amount del navegador	Servidor ignora el amount del cliente
P03	Manipular plan/application	Servidor conserva lo asociado a CheckoutSession
P04	Capture COMPLETED valido	Provisioning correcto una sola vez
P05	Repetir mismo orderId	Idempotente; sin duplicados
P06	Repetir mismo captureId	Idempotente; sin segundo Payment
P07	Capture PENDING	No fulfillment final
P08	Capture DENIED	No provisioning
P09	Webhook firma invalida	401/400 o equivalente; 0 side effects
P10	Webhook valido repetido	0 duplicados
P11	Fallo DB en paso intermedio	Rollback o estado recuperable; nunca falso exito
P12	`npm run build`	OK sin errores
P13	Tests de seguridad/checkout	Todos los tests criticos pasan

14. Puerta de produccion de esta fase

FENIXCMS_1 se considera terminada solo cuando se cumplan simultaneamente estas condiciones:

- La venta publica ya no llama a provisioning como si el POST fuera una confirmacion de pago.
- PayPal Order y Capture funcionan con llamadas reales de servidor y credenciales separadas por entorno.
- El backend es la autoridad sobre producto, importe, moneda, tenant y entitlements.
- El provisioning es idempotente y no deja exito falso ante errores.
- Webhook verification es criptograficamente valida y la idempotencia se persiste en PostgreSQL.
- Las pruebas de ataque directo y manipulacion de body estan automatizadas.
- Build y tests reales quedan registrados en el reporte de AI Studio.
- No se despliega al VPS hasta que todos los puntos anteriores esten demostrados.

15. Dependencias para fases posteriores

Una vez cerrada esta fase, las siguientes prioridades naturales son proteger el checkout de productos de la tienda (precio/stock/tenant), almacenamiento real de media y paquetes, autenticacion/CSRF/rate limiting distribuido, eliminacion completa de fallbacks de produccion, CI/CD y observabilidad.

Esas tareas no deben mezclarse con FENIXCMS_1 salvo que una dependencia inmediata impida cerrar la seguridad del pago.

****FIN DEL DOCUMENTO FENIXCMS_1****

Este documento define la Fase 1 de remediacion para preparar la venta publica de FenixCMS. La evidencia de cierre debe ser el reporte real de AI Studio + tests + build + migraciones + revision de codigo.